

SLA 约束下异构边缘 LLM 分层部署研究

整体研究路线与当前 Conservative Evaluator 思路

SLA-aware Evaluator → Layer-Level Deployment → Realistic Workload Evaluation
→ Good Throughput

1. 整体研究目标与研究路线

我目前希望研究的是：在异构 GPU 节点和受限物理网络中，如何进行 Layer-Level LLM 分层部署，使系统在给定 TTFT / TPOT SLA 和 workload 下获得更高的 good throughput。

最终关心的是在真实动态请求事件到达下的 good throughput 比例，即实际服务过程中满足 TTFT / TPOT SLA 的 token 占全部服务 token 的比例。

第一阶段：固定 Pipeline Evaluator

给定一条已经确定的 Layer-Level pipeline，快速估计在当前 SLA 和 workload 模型下，这条 pipeline 能够保守、安全地支持多大的请求速率。Evaluator 输出的是 Conservative SLA-Safe Request Capacity。这个 request-rate capacity 不是最终研究目标，而是后续 deployment search 中用于评价候选方案的内部指标。

核心思想：把 SLA 映射为资源需求 / 资源开销。 不把 TTFT / TPOT 只作为部署后的后验性能指标，而是根据当前请求状态和 profiling，先得到 SLA 中剩余的可用时间预算，再反推出为满足 SLA 所需的最低网络服务能力，并结合动态显存约束形成 pipeline 的资源可行性评价。

第二阶段：SLA-Aware Layer-Level Deployment

Deployment Algorithm 生成不同的 Layer-Level pipelines，并调用 Evaluator 对候选方案进行评价。算法利用这一 SLA-aware capacity information 寻找更优的 Layer-Level deployment，而不是仅依据静态 GPU、memory 或 bandwidth capacity 进行部署。

第三阶段：Good Throughput Evaluation

最终得到 deployment 后，再使用更真实的 workload trace / serving simulation 对不同方案进行统一评价。在相同物理资源、相同 workload 和相同 TTFT / TPOT SLA 下，比较各方案最终能够获得的 good throughput。

后续实验计划将本文方法与 Helix 等已有异构 GPU / network-aware Layer-Level deployment 方法作为 baseline 进行比较。

Request-rate Capacity 是搜索阶段的评价标准；Good Throughput 才是最终研究结果。

2. 为什么需要单独设计 Pipeline Evaluator

已有 Layer-Level deployment 可以根据 GPU 算力、显存、网络带宽等资源决定模型如何切分和部署。但如果最终目标是 SLA 下的 good throughput，仅仅知道一个 pipeline 的静态资源容量还不够。

原因是 LLM serving 中请求状态不断变化：Prefill 和 Decode 的计算特征不同；Prefill / Decode 并发数量不断变化；Decode context 持续增长；context 增长会改变计算时间；计算时间变化又会改变 TPOT 中能够留给网络传输的时间；KV Cache 也会随请求数量和 context 动态增长。

因此，同一条 Layer-Level pipeline 在不同请求状态下能够安全支持的负载并不是一个固定静态值。我的想法是先建立一个面向固定 Layer-Level pipeline 的低成本 SLA-aware capacity evaluator，再用它作为后续 deployment algorithm 的评价基础。

3. Evaluator 的核心思路：从 SLA 反推最低网络服务需求

目前最核心的思路不是先假定资源分配，再模拟最后 latency 是否满足 SLA，而是根据当前请求状态，先判断 TTFT / TPOT 在扣除计算以后还剩多少时间，再反推物理网络至少需要提供多少服务能力。

对于当前请求，根据 GPU 类型、Pipeline 中的 Layer Stage、当前 Prefill / Decode 并发状态以及 Prompt / Context Length，利用 profiling 或近似 performance model 得到计算时间。然后从 TTFT 或 TPOT budget 中扣除计算时间及其他必要开销。

如果计算部分已经耗尽 SLA budget，则即使网络传输时间为零，当前状态仍然无法满足 SLA。如果还有剩余时间，则根据 Stage Boundary 需要传输的数据以及实际经过的物理路径，反推出为了满足当前 SLA，各物理链路至少需要为该请求提供多大的服务能力。

Request State → Compute Profiling → Remaining SLA Time → Minimum Network Service Requirement → Physical-Link Feasibility

当多个 active requests 经过同一条物理链路时，对其最低服务需求进行保守聚合，并与实际链路容量比较。因此这里评价的不是 runtime 此刻实际分到了多少带宽，而是在当前请求状态下，如果希望这些请求都安全满足 SLA，当前 Layer-Level deployment 至少需要什么样的网络服务能力。

4. 请求状态采用轻量事件驱动评价

Evaluator 的目的不是重新实现一个完整的高保真 LLM runtime simulator，因为它未来需要被 deployment algorithm 大量调用。因此第一版只处理会显著改变资源状态的关键事件：

- Arrival
- Prefill → Decode
- Decode Context Block Update
- Finish

Prefill 暂时作为一个整体计算阶段。Decode 采用“连续计算进度 + 离散资源状态更新”：两个事件之间，根据当前 profiling 得到的 Decode speed 连续推进请求进度，但不逐 token 重新计算资源。

例如每生成约 16 个 token 后触发一次 Decode Block Update，再重新计算 Context-dependent compute time、SLA remaining budget、Network service requirement 和 KV Cache memory。这样希望在保留主要动态特征的同时，显著降低 evaluation cost。

5. Compute、Memory 与 Network 的处理方式

Compute

GPU compute 不再简单表示成固定需求相加，而主要通过 profiling 转化成 Prefill execution time 和 Decode per-token time，并直接消耗 TTFT / TPOT budget。Profiling 可以来自实际测量、公开数据，也可以结合模型结构、GPU compute throughput 和 memory bandwidth 建立近似模型。

Memory

显存继续作为显式硬约束，包括模型权重、KV Cache、workspace 和安全余量。尤其 KV Cache 会随 active requests 和 context 动态变化。

Network

网络是当前重点关注部分。网络需求并非预先固定，而是通过 Request State → Compute Time → Remaining SLA Budget → Required Network Service 动态产生。同时 Layer-Level placement 又决定 Stage Boundary 的 activation 实际经过哪些物理链路。

因此目前重点研究的是 LLM Request State、SLA 与 Deployment-Induced Physical Network Resource 之间的动态耦合。

6. Workload 与 Evaluator 输出

第一版使用有限 workload trajectory。请求负载经历低负载、负载上升、高负载、负载下降、Arrival 停止以及 Drain。Arrival 停止后仍继续推进，直到所有 active requests 完成。

Evaluator 记录整个过程中的 Prefill / Decode 数量、Decode progress 和 context、各链路最低服务需求和剩余带宽、GPU memory occupation, 以及 SLA / network / memory violation 的发生状态。

然后逐步提高请求负载强度。如果整个 workload trajectory 都没有突破当前 SLA / resource red line, 则认为该请求速率对当前 pipeline 是 safe 的, 最终得到该 pipeline 在当前保守评价规则下能够支持的最大 SLA-safe request rate。

这个值用于比较不同 candidate pipelines, 而不是直接作为论文最终性能指标。

7. Evaluator 在整体研究中的位置

因此目前这个 Evaluator 更准确的定位是 Deployment Search Oracle / Evaluation Criterion。它回答的是: “对于这个 candidate Layer-Level pipeline, 我应该给它多高的 SLA-safe capacity score?” 而不是整篇论文最终的: “这个系统真正获得了多少 good throughput?”

后一个问题需要在 deployment 已经确定以后, 通过统一、更真实的 workload evaluation 得到。

Search Phase: Candidate Pipeline → Conservative Evaluator → SLA-Safe Request Capacity

Final Evaluation: Selected Deployment → Realistic Workload / Serving Evaluation → Good Throughput

如果 Evaluator 能够以远低于完整 serving simulation 的成本, 对 candidate pipelines 的 SLA 服务能力进行有效排序, 那么它就能够帮助 Layer-Level deployment algorithm 在较大的搜索空间中进行优化。

8. 后续算法与 Baseline

Evaluator 完成后, 下一步是设计 SLA-aware Layer-Level deployment algorithm。最终实验中计划将本文方法与已有 deployment 方法进行比较, 例如以 Helix 这一类异构 GPU / network-aware Layer-Level deployment 方法作为重要 baseline。

比较时不只看各算法自身原来的优化目标, 而是把不同算法产生的 deployment 放到相同基础设施、相同 workload 和相同 TTFT / TPOT SLA 下统一执行最终评价。

- Good throughput
- TTFT / TPOT SLA attainment
- Network bottleneck / utilization
- GPU memory feasibility

- 不同负载下的 robustness

希望最终回答的问题是：相比主要面向 throughput 或静态资源容量的 Layer-Level deployment，显式利用 SLA-aware pipeline capacity information，是否能够找到在动态 workload 下获得更高 good throughput 的部署方案。

9. 当前阶段与下一步

目前主要完成的是整个研究路线中的第一部分：固定 Layer-Level Pipeline 的 Conservative SLA-Safe Evaluator。

当前希望先确认：

- 以这种 Evaluator 作为后续分层部署评价标准是否合理；
- “SLA remaining time $\rightarrow$ minimum network service requirement” 这一网络侧核心逻辑是否成立；
- 是否存在可以进一步借鉴的网络 QoS / SFC / resource reservation 理论。

如果主线合理，下一步准备冻结第一版问题定义，开始实现最小 prototype，并通过实验验证 Evaluator 是否能够低成本、有效地评价不同 candidate pipelines。

一句话概括

最终目标是提高 SLA 约束下 Layer-Level LLM deployment 的 good throughput；当前首先设计一个面向固定 pipeline 的低成本 Conservative Evaluator，将 SLA 映射为当前状态下的资源需求：根据 Prefill / Decode 请求状态和 profiling 得到计算时间，再从 TTFT / TPOT 剩余预算反推出物理网络最低服务需求，并结合动态 KV 显存约束得到 SLA-safe request capacity；后续以这一 capacity 作为 deployment search 的评价标准，并与 Helix 等已有方法在统一 workload 和 SLA 下比较最终 good throughput。